

Student 6 **DAYYAM JEEVANA REDDY** 192411259RD

5 / 5 submitted

Q1. ATM Withdrawal – Exception Handling Debugging

CODE

```
import java.util.Scanner;
class ATM{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int balance =sc.nextInt();
        int amount =sc.nextInt();
        try{
            if(amount<=0){
                int triggerException = balance/amount;
            }
            if(amount>balance){
                System.out.println("Insufficient Balance");
            }else{
                System.out.println("Withdrawal successful");
                System.out.println("Remaining balances"+(balance-amount));
            }
        }catch(ArithmeticException e){
            System.out.println("Transaction failed");
        }finally{
            System.out.println("Transaction completed");
        }
        sc.close();
    }
}
```

INPUT

5000
2000

OUTPUT

Withdrawal successful
Remaining balances3000
Transaction completed

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.Scanner;
class PatientRegistration{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String inputName = sc.nextLine();
        String name;
        if(inputName.equals("NULL")){
            name=null;
        }else{
            name=inputName;
        }
        int age = sc.nextInt();
        if(name==null){
            System.out.println("PatientName: null");
        }else{
            System.out.println("Patient name:"+name.toUpperCase());
        }
        if(age>=18){
            System.out.println("Adult patient");
        }else{
            System.out.println("Minor Patient");
        }
        System.out.println("Registration completed");
        sc.close();
    }
}
```

INPUT

Jhon Doe
25

OUTPUT

Patient name:JHON DOE
Adult patient
Registration completed

Q3. E-Commerce Product Search – Array Exception Debugging

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;
class ProductSearch{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] products = {
            "Laptop",
            "Mobile",
            "Tablet",
            "Headphone",
            "Keyboard"
        };
        try{
            int productNumber= sc.nextInt();
            System.out.println("Selected product:"+products[productNumber-1]);
        }catch(InputMismatchException e){
            System.out.println("Invalid input!Please enter a number.");
        }catch(ArrayIndexOutOfBoundsException e){
            System.out.println("Product not found! Choose a number between 1 and 5.");
        }finally{
            System.out.println("Search completed");
        }
        sc.close();
    }
}
```

INPUT

1

OUTPUT

Selected product:Laptop
Search completed

Q4. Railway Reservation – Synchronization Debugging

CODE

```
class RailwayReservation{
    public static void main(String[] args){
        Reservation r=new Reservation();
        Customer c1=new Customer(r,"customer 1");
        Customer c2=new Customer(r,"Customer 2");
        c1.start();
        c2.start();
    }
}
class Reservation{
    int seats = 1;
    synchronized void bookSeat(String customer){
        if(seats>0){
            System.out.println(customer+"is booking a seat");
            try{
                Thread.sleep(100);
            }catch(Exception e){
                System.out.println("Thread interrupted");
            }
            seats--;
            System.out.println(customer + "booking successfull");
        }else{
            System.out.println(customer+"No seat available");
        }
    }
}
class Customer extends Thread{
    Reservation reservation;
    String name;
    Customer(Reservation r,String name){
        reservation = r;
        this.name = name;
    }
    public void run(){
        reservation.bookSeat(name);
    }
}
```

OUTPUT

```
customer 1is booking a seat
customer 1booking successfull
Customer 2.No seat available
```

Q5. Bank Account – Race Condition Debugging

CODE

```
class BankAccount{
    int balance = 10000;
    synchronized void withdraw(int amount){
        if(balance>= amount){
            System.out.println(Thread.currentThread().getName()+"Withdrawing"+amount);
            try{
                Thread.sleep(100);
            }catch(Exception e){}
            balance-=amount;
            System.out.println("Withdrawal successful");
        }else{
            System.out.println("Insufficient balance");
        }
    }
}

public static void main(String[] args)
    throws Exception {
    BankAccount a = new BankAccount();
    ATM t1 = new ATM(a,7000);
    ATM t2 = new ATM(a,6000);
    t1.setName("ATM-1");
    t2.setName("ATM-2");
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println("Final Balance ="+ a.balance);
}

class ATM extends Thread{
    BankAccount a;
    int amount;
    ATM(BankAccount a, int amount){
        this.a =a;
        this.amount = amount;
    }
    public void run(){
        a.withdraw(amount);
    }
}
```

OUTPUT

```
ATM-1Withdrawing7000
Withdrawal successful
Insufficient balance
Final Balance =3000
```